

Laboratorio 4 — AMQP + tercer protocolo (HTTPS) frente a MQTT, hacia Azure IoT Central

IoT + Cloud + Sistemas Distribuidos · UNAB · Escenario UNAB-Ambiental · App IoT Central: lab4-medidordeclima-unab · Repositorio: <https://github.com/Daniverd15/Laboratorio-4>

1. Qué se hizo y rol de AMQP en Azure

Se publicaron las **mismas 3 variables** (temperature, humedad, illuminance; 53 B de JSON) a la **misma app de IoT Central** por tres transportes desde una VM Azure (Ubuntu, Python 3.10): **MQTT** (línea base del Lab 3), **AMQP 1.0** (Etapa 1) y **HTTPS/REST** (Etapa 2, tercer protocolo). AMQP 1.0 es el protocolo del **plano de servicio** de Azure (Service Bus, Event Hubs; el endpoint compatible de IoT Hub se lee por AMQP): conexión → sesión → **enlaces con crédito** (control de flujo) y **liquidación** por mensaje (accepted/rejected/released), frente al pub/sub ligero de MQTT. IoT Hub también lo ofrece al dispositivo (5671, o 443 por WebSockets). La suscripción de los Labs 1–3 se quedó sin créditos: se **recreó** la app, la plantilla Hobo MX-100 v3, la VM y los dispositivos en una suscripción nueva.

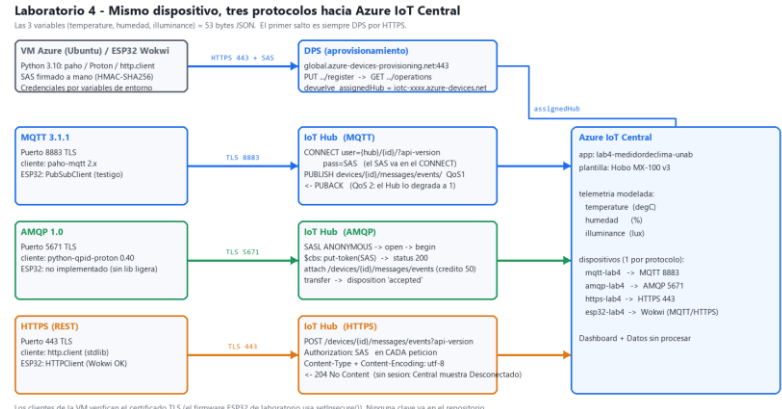


Fig. 1 — Tres caminos hacia IoT Central; DPS por HTTPS en todos.

Etapa 1 — AMQP hacia IoT Central (checkpoint cumplido)

- **Ciente:** Apache Qpid Proton 0.40. El SDK azure-iot-device de **Python** no tiene AMQP (0 archivos lo mencionan; su websockets es MQTT-WS).
- **Flujo:** TLS 5671 verificado + SASL ANONYMOUS → open/begin → nodo \$cbs: put-token con el **SAS firmado a mano** (HMAC-SHA256) → status-code 200 (10 ms) → enlace /devices/{id}/messages/events con **crédito 50** → transfer → disposition accepted (≈103 ms).
- **Anotado (lo que pide el lab):** puerto 5671 con TLS verificado; sí **aparece** en datos sin procesar (Fig. 2), dashboard (Fig. 3) y Explorador de datos (Fig. 5), dispositivo *Conectado*. **Qué cambia vs MQTT:** conexión persistente en ambos, pero AMQP añade **sesión + 3 enlaces** (\$cbs x2 y telemetría); tamaño percibido **438 B vs 365 B** por mensaje (+20 %); ack por mensaje; **SAS renovable en caliente** (TTL 120 s: 3 renovaciones en 330 s, **0 reconexiones**); debug por tramas (PN_TRACE_FRM, Fig. 6).
- **Qué resuelve mejor que MQTT:** contrapresión por crédito, liquidación explícita y seguridad en banda. **No** gana en latencia ni en bytes (Sec. 3).
- **Tropiezos reales:** el wheel de Proton para Linux **no trae TLS** (se compiló contra OpenSSL); Proton entrega objetos nuevos por evento (los enlaces se comparan por nombre).

2. Etapa 2 — Tercer protocolo: HTTPS (REST) hacia IoT Hub / Central

Justificación de negocio: un sensor de batería que despierta cada pocos minutos paga más por mantener una sesión que por una petición, y HTTPS (443) atraviesa los firewalls/proxys del cliente sin abrir puertos. **Implementación** (https_telemetria.py, solo stdlib, 117 líneas): POST https://{hub}/devices/{id}/messages/events con Authorization: SAS (firmado a mano), Content-Type: application/json y Content-Encoding: utf-8 → 204. Hallazgo: sin ese último encabezado el Hub acepta el POST pero **Central no interpreta el JSON**. Depurable con curl (204 válido / 401 IotHubUnauthorizedAccess con firma alterada). También se simuló en el **ESP32 (Wokwi)**: DPS por REST + POST con HTTPClient (1.er POST ≈ 4,0 s por el handshake TLS del ESP32; siguientes ≈ 162 ms) y el testigo MQTT (TCP+TLS+CONNECT ≈ 3,8 s); ambos llegan a Central (Fig. 8). **Checkpoint (log + captura):** log del cliente en la VM con 204 en cada envío (Fig. 7) y la telemetría en Central (Fig. 4).

Ventajas

- Sin librería ni broker; **117 líneas**; depurable con curl y códigos de estado estándar.
- 443 pasa casi cualquier firewall/proxy.
- Funciona en el ESP32 simulado (HTTPClient).

Desventajas (medidas; † = documentación)

- **901 B/msg** (17×); sin keep-alive **5,7 KB y 199 ms** por mensaje.
- El SAS viaja en **cada** petición; Central lo muestra **Desconectado** (sin presencia, Fig. 4).
- Sin comandos hacia el dispositivo† (habría que sondear mensajes C2D).

3. Mediciones (VM Azure → IoT Hub; N=51 mensajes de 53 B; bytes IP totales con contadores iptables)

Protocolo	Setup hasta enviar	ACK p50 / p95	Bytes/msg en cable	Conexión + 1 msg	Paq. /msg	Corte 16 s (reset): perdidos · 1.er ACK tras volver la red	Corte 16 s (drop)
MQTT 8883 QoS 1	45 ms	103 / 118 ms	365 (6,9×)	6 180 B	4,0	0 · 1,5 s · paho reencola solo	0 pérd.; sin reconectar (TCP retransmite), 0,3 s
MQTT 8883 QoS 0	50 ms	— (sin ack)	221 (4,2×)	6 129 B	1,9	(QoS 0 no garantiza entrega)	—
MQTT-WS 443 QoS 1	166 ms	105 / 113 ms	370 (7,0×)	11 922 B	4,0	—	—
AMQP 5671	100 ms	103 / 111 ms	438 (8,3×)	9 162 B	4,2	0 · 1,6 s · 1 reenvío (posible duplicado)	0 pérd.; sin reconectar , 0,2 s
HTTPS 443 keep-alive	41 ms	109 / 115 ms	901 (17×)	5 730 B	5,0	0 · 1,3 s · cola y reintentos en la app	0 pérd.; reconecta tras timeout de 20 s, 5,3 s
HTTPS 443 sin keep-alive	35 ms	199 / 236 ms	5 699 (107×)	5 691 B	16,1	—	—

Lectura: la **latencia no distingue** a MQTT, AMQP y HTTPS con keep-alive (el ack tarda ≈100 ms en los tres, mientras un put-token AMQP se contesta en 10 ms: domina el Hub); lo que cambia es el **overhead** (MQTT < AMQP < HTTPS), el costo de **abrir** la conexión (AMQP hace SASL+open+begin+CBS+attach ≈ 5 viajes) y **quién** garantiza la entrega (paho en MQTT QoS 1; la aplicación en AMQP/HTTPS: cola en memoria + backoff ≤ 2 s). Con AMQP hay que **deduplicar por message-id**. En MQTT el SAS solo existe en el CONNECT: en 330 s con TTL 120 s el Hub **no** cortó la conexión, pero no debe confiarse en eso. Evidencia de las mediciones: evidencias/05 (bytes y latencia), 06 (corte de red), 07 (renovación del SAS) y los CSV/logs de mediciones/.

4. Repositorio entregable y límites

Repo: <https://github.com/Daniverd15/Laboratorio-4> — amqp/amqp_iotHub.py (script AMQP en la VM), https/https_telemetria.py (tercer protocolo), mqtt/ (línea base), wokwi/ (ESP32 MQTT y HTTPS), medicion/ (benchmarks y corte de red), README.md (dependencias y cómo reproducir, **sin secretos**: claves por variables de entorno) y /evidencias (logs, tramas AMQP, capturas de Central y de Azure). **Límites:** una VM/región y N=51 por protocolo; AMQP no se probó en el ESP32 ni sobre WebSockets; el firmware ESP32 usa setInsecure() (simplificación de laboratorio); las latencias (~100 ms) están dominadas por el Hub, así que no se afirma ventaja de latencia de ningún protocolo.

5. Evidencia en IoT Central y en la VM (misma app lab4-medidordeclima-unab, plantilla Hobo MX-100 v3)

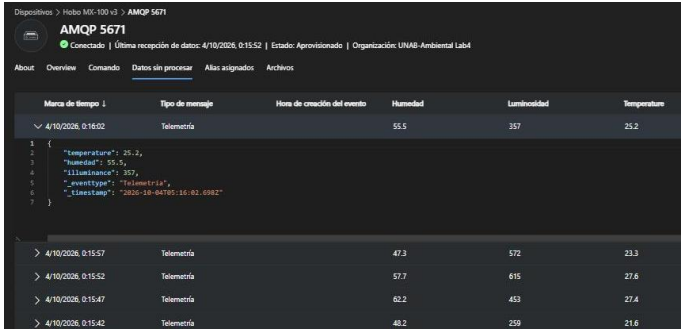


Fig. 2 — AMQP: datos sin procesar con el JSON expandido (temperature, humedad, illuminance); dispositivo Conectado.

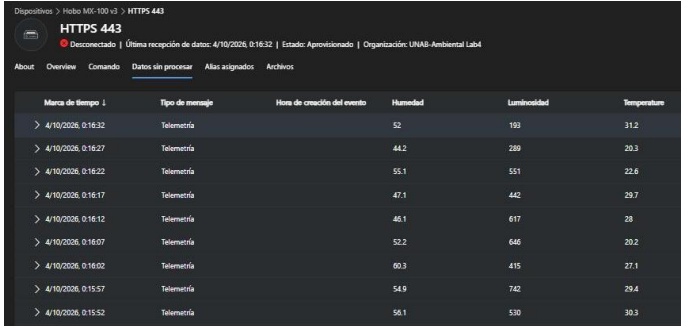


Fig. 4 — HTTPS: la telemetría llega cada 5 s, pero el dispositivo figura Desconectado (no hay sesión).

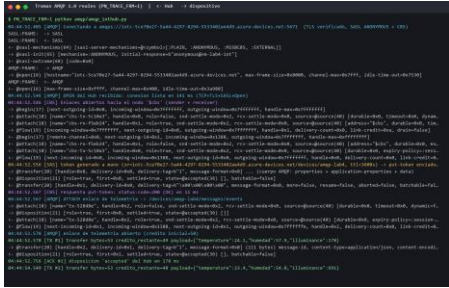


Fig. 6 — Tramas AMQP reales (PN_TRACE_FRM): SAS, open, begin, attach \$cbs, flow (crédito 50), put-token → accepted, attach de telemetría (completo: evidencias/02b).

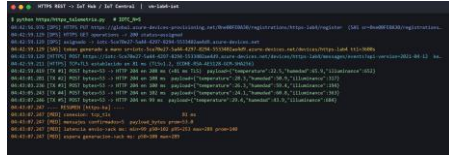


Fig. 7 — Log del cliente HTTPS en la VM: DPS por REST, POST y HTTP 204 por cada lectura (curl: evidencias/11).

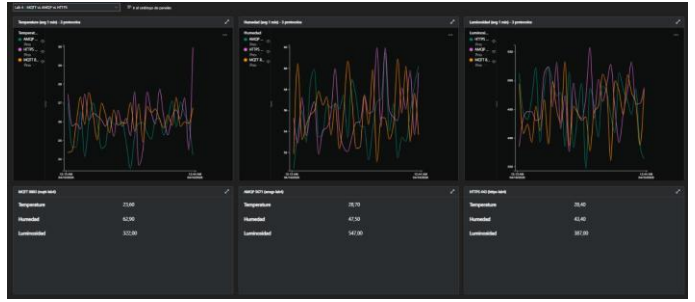


Fig. 3 — Dashboard: las 3 variables de MQTT, AMQP y HTTPS en la misma app (gráficas y último valor).



Fig. 5 — Explorador de datos: Temperature por dispositivo (los 3 protocolos + ESP32).

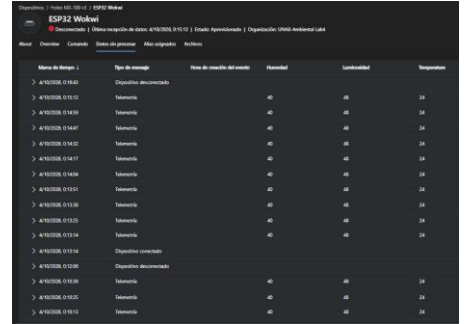


Fig. 8 — ESP32 en Wokwi en Central: HTTPS (sin eventos de conexión) y MQTT (conectado/desconectado).

Las capturas de Central muestran hora de Bogotá (UTC-5); los logs de la VM, UTC (p. ej. 0:15 en Central = 05:15 en el log).

6. Tabla comparativa MQTT · AMQP · HTTPS (†: de la documentación, no medido aquí)

Criterio	MQTT 3.1.1 (8883)	AMQP 1.0 (5671)	HTTPS REST (443)
Overhead por mensaje (payload 53 B)	365 B (QoS 0: 221)	438 B	901 B (sin keep-alive 5 699)
Latencia percibida (ACK p50)	103 ms	103 ms	109 ms (199 sin keep-alive)
Fiabilidad	QoS 0/1 (QoS 2 se degrada a 1, Lab 3); paho reencola	accepted/rejected/released por mensaje; reenvío por la app	Solo lo que programe la app (cola, reintentos, timeouts)
Renovar el SAS	Reconectar	En caliente (CBS) — medido	Un SAS nuevo en cada petición
Microcontrolador / Wokwi	Probado ESP32 (PubSubClient; TLS 3,8 s)	No implementado: sin cliente ligero estándar en Arduino-ESP32 (el SDK C de Azure sí, pesado†)	Probado ESP32 (HTTPClient; 4,0 s + 162 ms)
Facilidad de debug	Media (ZOTC_DEBUG: paquetes crudos)	Baja—media (PN_TRACE_FRM: estructuras + payload binario)	Alta (curl -i, códigos estándar)
Paso de firewalls	8883 a veces bloqueado → MQTT-WS 443 probado (370 B/msg)	5671; AMQP-WS 443† (Proton Python no lo expone)	443 , atraviesa proxys
Comandos / estado en Central	Métodos directos: setAlertLed → 200 (medido) · <i>Conectado</i>	Comandos: sí† · estado: <i>Conectado</i> (medido)	Comandos: no† · estado: <i>Desconectado</i> (medido)
Complejidad real (líneas de código)	130	255 + compilar Proton con TLS en Linux	117
Caso de uso ideal	Telemetría continua de dispositivos	Plano de servicio, pipelines con contrapresión, gateways†	Sensores que duermen minutos; integración y diagnóstico

7. Recomendaciones para la plataforma propia (Labs 5–8)

- Dispositivo (Wokwi / ESP futuro):** MQTT sobre TLS 8883, QoS 1. Menos bytes (365 vs 438/901 B), presencia (*Conectado*), comandos y librería probada en el ESP32; renovar el SAS **reconectando antes de que venza**. Plan B tras firewalls estrictos: **MQTT-WS 443** (mismo modelo, mismos bytes por mensaje).
- Entre servicios Azure y en la plataforma propia: AMQP 1.0** (Service Bus / Event Hubs): sin penalización de latencia (103 ms), con crédito para **contrapresión**, liquidación por mensaje y renovación de credenciales sin cortar el flujo; consumidores **idempotentes** (clave message-id).
- Nos quedamos con dos protocolos: MQTT (borde) + AMQP (servicios).** Cada protocolo extra suma superficie de autenticación, de depuración y de código (AMQP ya costó 255 líneas y compilar Proton). HTTPS queda como **herramienta** (diagnóstico con curl, webhooks, dispositivos que duermen minutos), no como transporte permanente: 901 B/msg, sin presencia y sin comandos.